

HDOC - Day 35

Variable-Length Square Array Operations

Algorithm

1. Read positive order n and declare VLA a[n][n]. 2. Read the square matrix. 3. Display the menu repeatedly. 4. Option 1 compares all main-diagonal elements a[i][i] to check distinctness. 5. Option 2 adds all a[i][i]. 6. Option 3 traverses diagonals having equal row+column index and alternates direction upward-right/downward-left. 7. Exit on option 4; show an error for invalid choices.

Test Case Table

TC	Input	Expected	Actual	Result
1	1 2 3 / 4 5 6 / 7 8 9; op 1	1,5,9 distinct	1,5,9 distinct	Pass
2	Same; op 2	Sum=15	Sum=15	Pass
3	Same; op 3	1 2 4 7 5 3 6 8 9	1 2 4 7 5 3 6 8 9	Pass
4	1 2 3 / 4 1 6 / 7 8 9; op 1	Not distinct	Not distinct	Pass

C Program

```
#include <stdio.h>
void distinct(int n,int a[n][n]);
void sumDiagonal(int n,int a[n][n]);
void traverse(int n,int a[n][n]);

int main() {
    int n,i,j,ch;
    do { printf("Enter order: "); scanf("%d",&n); } while(n<=0);
    int a[n][n];
    printf("Enter elements: ");
    for(i=0;i<n;i++) for(j=0;j<n;j++) scanf("%d",&a[i][j]);
    do {
        printf("\n1. Distinct main diagonal\n2. Main diagonal sum\n");
        printf("3. Diagonal traversal\n4. Exit\nEnter choice: ");
        scanf("%d",&ch);
        switch(ch) {
            case 1: distinct(n,a); break;
            case 2: sumDiagonal(n,a); break;
            case 3: traverse(n,a); break;
            case 4: break;
            default: printf("Invalid choice.\n");
        }
    } while(ch!=4);
    return 0;
}

void distinct(int n,int a[n][n]) {
    int i,j,ok=1;
    printf("Main diagonal: ");
    for(i=0;i<n;i++) printf("%d ",a[i][i]);
    printf("\n");
    for(i=0;i<n;i++)
        for(j=i+1;j<n;j++)
            if(a[i][i]==a[j][j]) ok=0;
    if(ok) printf("All main diagonal elements are distinct.\n");
    else printf("Main diagonal elements are not distinct.\n");
}

void sumDiagonal(int n,int a[n][n]) {
    int i,sum=0;
    for(i=0;i<n;i++) sum+=a[i][i];
    printf("Sum of main diagonal elements = %d\n",sum);
}

void traverse(int n,int a[n][n]) {
    int s,i,j,start,end;
    printf("Diagonal traversal: ");
    for(s=0;s<=2*n-2;s++) {
        start=(s<n)?0:s-n+1;
        end=(s<n)?s-n-1;
        if(s%2==1)
            for(i=start;i<=end;i++){j=s-i;printf("%d ",a[i][j]);}
        else
            for(i=end;i>=start;i--){j=s-i;printf("%d ",a[i][j]);}
    }
    printf("\n");
}
```

Compilation, Execution & Testing

```
puppie@Anvikas-MacBook-Air Desktop % clang day35.c -o day35
puppie@Anvikas-MacBook-Air Desktop % ./day35
Enter order: 3
Enter elements: 1 2 3 4 5 6 7 8 9
Enter choice: 1
Main diagonal: 1 5 9
All main diagonal elements are distinct.
```

```
puppie@Anvikas-MacBook-Air Desktop % ./day35
Enter order: 3
Enter elements: 1 2 3 4 5 6 7 8 9
Enter choice: 2
Sum of main diagonal elements = 15
```

```
puppie@Anvikas-MacBook-Air Desktop % ./day35
Enter order: 3
Enter elements: 1 2 3 4 5 6 7 8 9
Enter choice: 3
Diagonal traversal: 1 2 4 7 5 3 6 8 9
```

```
puppie@Anvikas-MacBook-Air Desktop % ./day35
Enter order: 3
Enter elements: 1 2 3 4 1 6 7 8 9
Enter choice: 1
Main diagonal: 1 1 9
Main diagonal elements are not distinct.
```

Result: The program was compiled, executed and tested successfully for distinct main-diagonal elements, diagonal sum and alternating diagonal traversal.